

Il concorrente artificiale

Una squadra di agenti che risolve problemi OII e insegna a migliorare il codice.
Costruita dal vivo con ADK 2.0 e Antigravity CLI.

IL METODO

Venerdì avete fatto tutti la stessa cosa

1

Leggi

testo, vincoli,
esempi

2

Stima

dai vincoli, la
complessità che
serve

3

Scrivi

la soluzione
efficiente

4

Prepara

brute force e
generatore

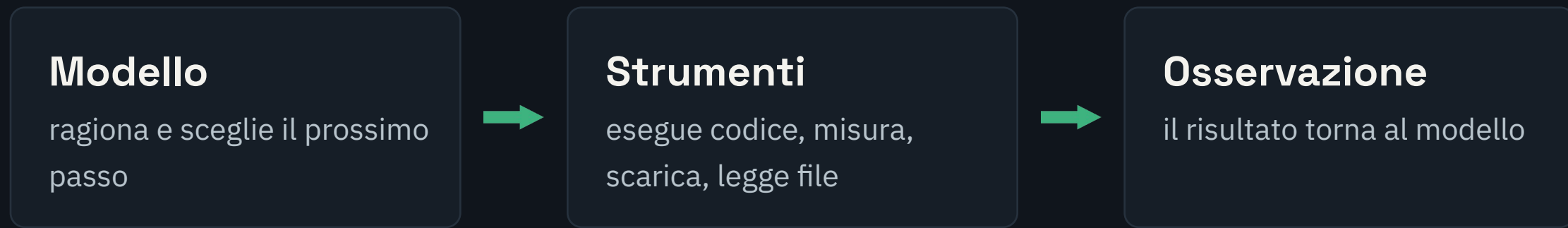
5

Confronta

finché non esce un
controesempio

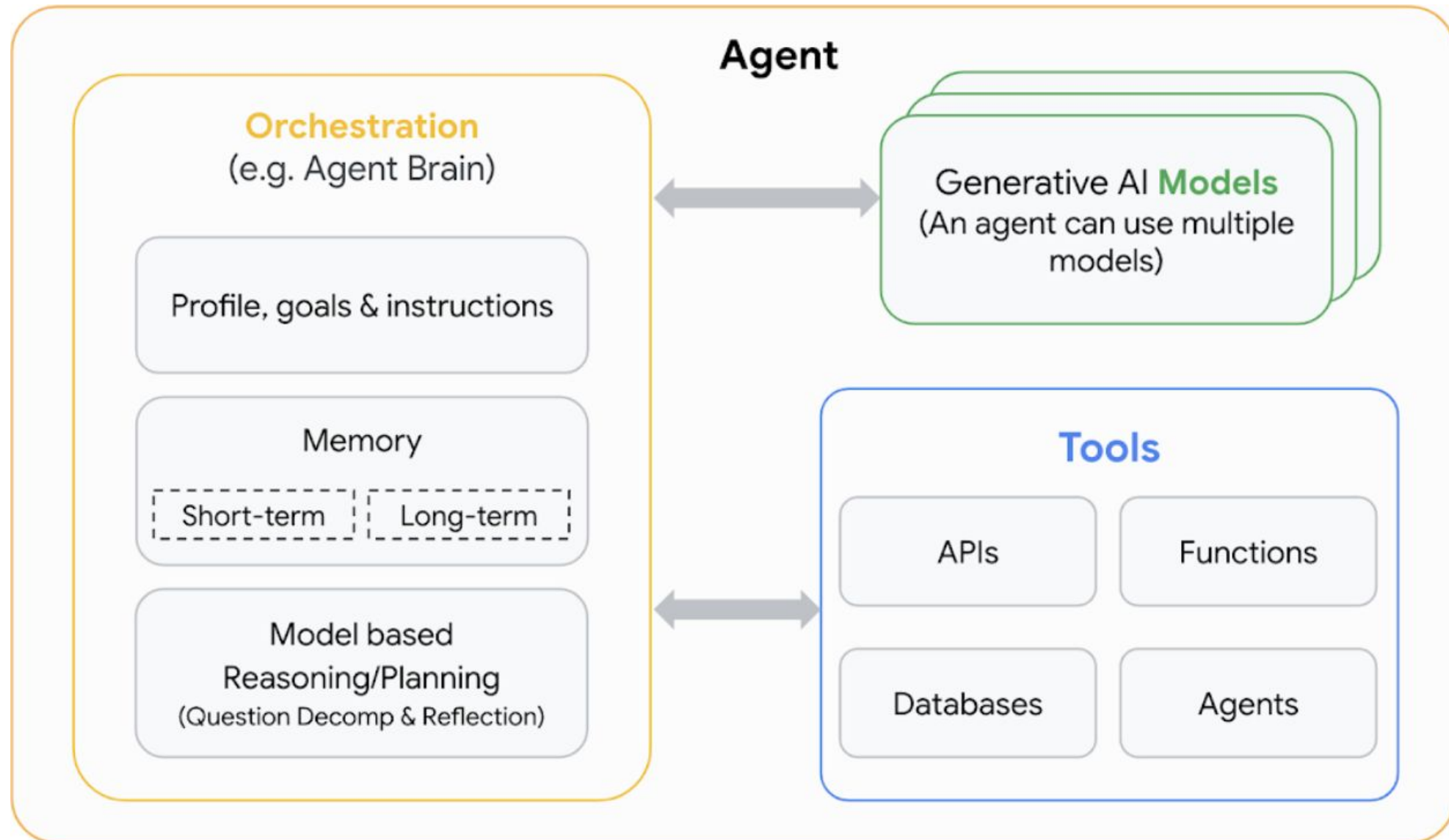
Oggi insegniamo questo metodo a una squadra di agenti.

Un agente è un modello dentro un ciclo



Lo stato tiene la memoria tra un passo e l'altro. Il ciclo si ferma quando il compito è finito, secondo la definizione che abbiamo stabilito.

What is an AI Agent?



Perché una squadra e non un solo agente

Specializzazione

Ogni agente ha un compito e un prompt corto. Chi legge il testo non scrive codice.

Errori indipendenti

Chi scrive i test non vede la soluzione: se sbagliano, sbagliano in modo diverso.

Un giudice che non è un LLM

Test e cronometro decidono. Gli agenti propongono soltanto.

ADK 2.0

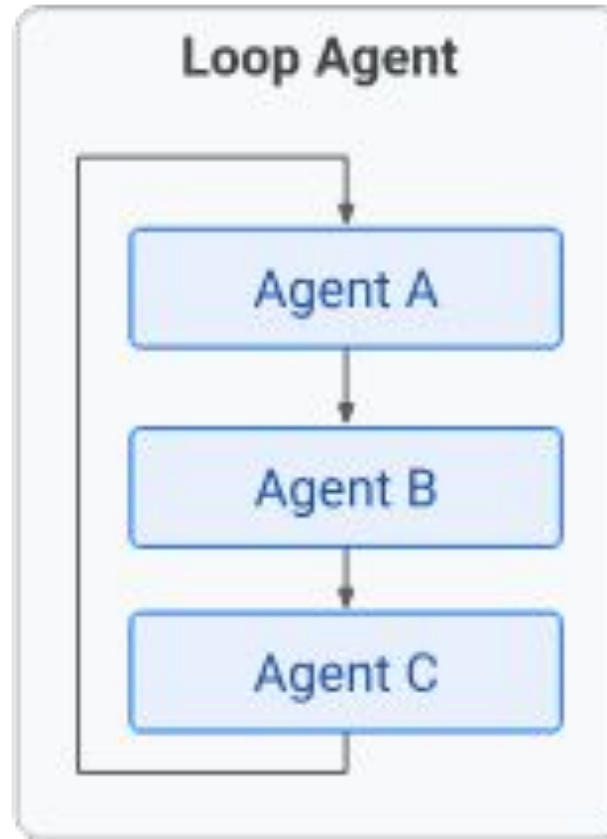
Un agente è un grafo

I grafi li conoscete meglio di molti ingegneri.

ADK 2.0 è stabile dal 19 maggio 2026.
Oggi usiamo la versione 2.9.

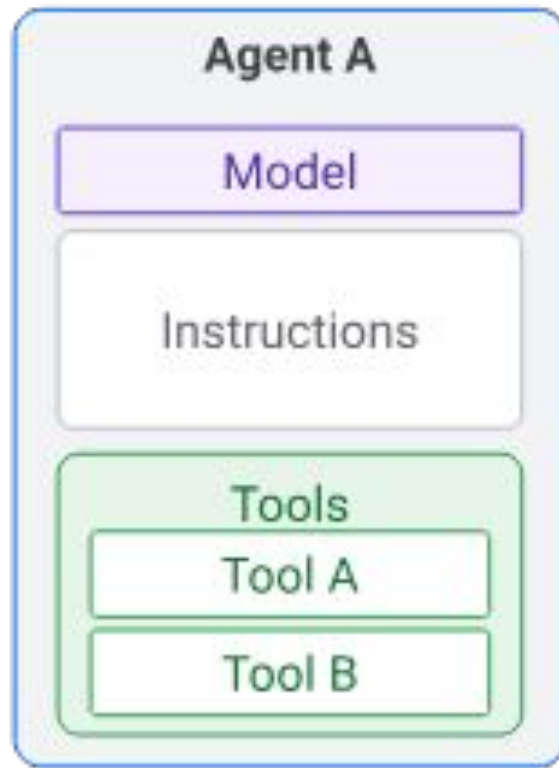
Primitiva	Cosa fa
Agent	nodo LLM con output tipizzato
Funzione	nodo di codice, deterministico
Arco	dopo questo nodo, quello
Fan-out + JoinNode	rami in parallelo, poi si aspetta
Route	arco condizionale: rework, done, stop
Ciclo	un arco che torna indietro
RequestInput	pausa finché un umano risponde

Agents workflows simple:

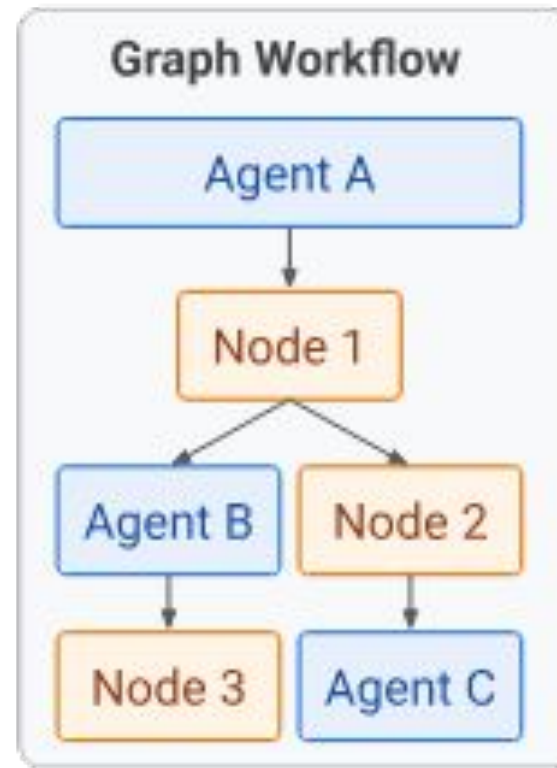


Agents workflows ADK2.x:

Simple Agent



Agent Workflows



IL PRINCIPIO

Gli LLM propongono. Il codice giudica.

Esempi, stress test e cronometro decidono se una soluzione è corretta e quanto vale.
Nessun agente si dà ragione da solo.

LA STRATEGIA

Prima corretta, poi veloce

v1 • Solver rapido

la soluzione più semplice
che sia corretta, in pochi
secondi



v2 ... vN • Reviewer

un miglioramento per volta,
misurato sui subtask



Stop

punteggio stimato pieno,
oppure MAX_STEPS passi

È la strategia di gara: prima i punti sicuri, poi si sale.

LA MODALITÀ DIDATTICA

Valuta il tuo codice

```
valuta oii_... + file .py
```

Il tuo codice diventa la v1: il sistema lo verifica contro la brute force, lo misura sui subtask e lo migliora un passo alla volta.

Ogni consiglio della guida è una versione che ha già superato i test.

```
# guida.md
## Review
corretto? · complessità · tabellone
## Passo 1
obiettivo · perché · cosa cambiare
come verificarlo · risultato misurato
## Passo 2 ...
## Soluzione finale
da guardare solo dopo averci provato
+ report.html
diff affiancati e grafico dei tempi
```

IL SISTEMA

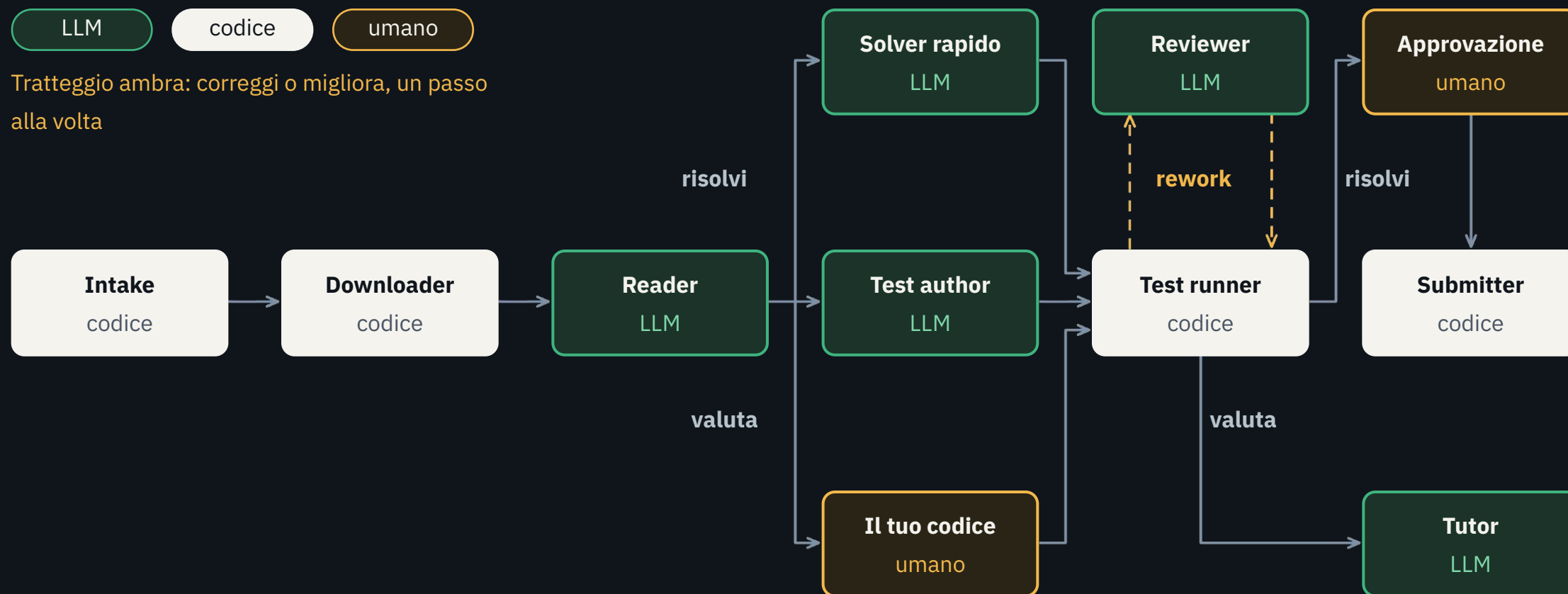
Il grafo, nodo per nodo

LLM

codice

umano

Tratteggio ambra: correggi o migliora, un passo alla volta



Chi fa cosa

Nodo	Tipo	Riceve	Produce
Intake e Downloader	codice	messaggio e nome del task	modalità, PDF, allegati, esempi
Reader	LLM	testo e allegati	specifica e subtask
Solver rapido	LLM	specifica	v1 semplice e corretta
Test author	LLM	specifica, non la soluzione	brute force e generatori
Test runner	codice	versione e test	correttezza, tempi, tabellone
Reviewer	LLM	codice e tabellone	un miglioramento alla volta
Tutor	LLM	storia delle versioni	review e guida passo passo
Approvazione	umano	tabellone finale	«invia» oppure stop
Submitter	codice	versione migliore	punteggio ufficiale

DAL VIVO

Prima del primo prompt: il contesto

I modelli conoscono soprattutto l'ADK 1.x.

Un file di contesto insegna all'agente la versione 2.x, l'API del sito, il grafo da costruire e le regole del gioco, prima di scrivere una riga.

```
# AGENTS.md
## Due modalità, un grafo
- risolvi <task> · valuta <task> + codice
## API ADK 2.x (verificate su 2.9.2)
- Workflow(edges=[...]), JoinNode, RequestInput
- NON usare SequentialAgent, LoopAgent
## API training.olinio.it
## Regole
- Gli LLM propongono, il codice giudica
- Python su stdin/stdout, una verifica per passo
```

DAL VIVO

Dieci prompt, quattro atti

Atto 1

Le fondamenta

1 • Scheletro e intake

risolvi e valuta

2 • Downloader

il task vero

3 • Reader

la tabella dei subtask

Atto 2

La prima
soluzione

4 • Solver // Test author

la v1 in pochi secondi

5 • Test runner

è corretta?

6 • Tabellone

quanto vale la v1

Atto 3

Migliora e invia

7 • Reviewer

un passo alla volta

8 • Invio

«invia» e grader ufficiale

Atto 4

Valuta il tuo
codice

9 • Valuta

review e guida

10 • Vista visiva

diff e tempi · facoltativo

IL RISULTATO

Il grafo in 12 righe

```
root_agent = Workflow(name="oii_solver", edges=[
    ("START", intake, download_task, reader, route_mode),
    (route_mode, {"risolvi": (quick_solver, test_author),
                  "valuta": test_author_v}),
    (quick_solver, join), (test_author, join), (join, test_runner),
    (test_runner, {"rework": reviewer, "done_solve": ask_approval,
                  "done_teach": tutor, "give_up": give_up}),
    (test_author_v, test_runner), (reviewer, test_runner),
    (ask_approval, handle_approval),
    (handle_approval, {"submit": submit, "stop": stop}),
    (tutor, render_report), (submit, render_report), (stop, render_report),
])
```

Ogni tupla è una catena di archi, ogni dizionario una route. La route "rework" verso reviewer è il ciclo.

Il ciclo sul controesempio

1. Lo stress test trova un input su cui la versione e la brute force non concordano.
2. Il più corto va al Reviewer, con entrambi gli output.
3. Il Reviewer corregge solo il bug; la nuova versione torna al Test runner.

SOMMA MASSIMA DI UN SOTTOARRAY NON VUOTO

```
best = cur = 0
for x in a:
    cur = max(x, cur + x)
    best = max(best, cur)
print(best)
```

Input	-5 -2 -7
Brute force	-2
Versione	0

Esempio illustrativo. La brute force prova tutti i sottoarray: lenta, ma non sbaglia. Con tutti valori negativi la risposta giusta è -2 , il sottoarray $[-2]$; la versione risponde 0 perché parte da `best = 0`, come se il sottoarray vuoto fosse ammesso. Il Reviewer corregge solo quella riga: `best =  $-\infty$` .

Due giudici diversi

	Grader deterministico	LLM come giudice
Decide	correttezza e tempi misurati	dove sta il collo di bottiglia e come spiegarlo
Stesso input, stesso verdetto	sì	non sempre
Punto debole	test troppo deboli	si lascia convincere da una spiegazione sbagliata
Nel nostro grafo	Test runner: esempi, stress, cronometro	Reviewer e Tutor, sempre dopo le misure

Un LLM giudica solo dove un test non può arrivare.

L'UMANO

L'ultima parola è umana

Il grafo si ferma con `RequestInput`. Senza «invia» non parte nessuna sottoposizione.

Account dedicato

Mai quello di un concorrente: i punteggi finiscono in classifica.

Prove a secco

`OLINFO_DRY_RUN=1` simula l'invio durante le prove.

Un tetto ai cicli

`MAX_STEPS=4` miglioramenti e `MAX_FIX=3` correzioni, poi si torna alla versione migliore.

IL RISULTATO

Com'è andata stamattina

Versione	Cosa è cambiato	Complessità	Stima
v1	[soluzione semplice]	[O(...)]	[__/100]
v2	[...]	[...]	[__/100]
v3	[...]	[...]	[__/100]
v4	[...]	[...]	[__/100]

Punteggio ufficiale: [__/100]

Dove sta andando la ricerca

AlphaEvolve • 2025

Un agente evolutivo basato su Gemini moltiplica matrici complesse 4×4 con 48 moltiplicazioni: l'algoritmo di Strassen del 1969 ne usava 49.

Kernel GPU • ICML 2025

Un ciclo agentico con Gemini 2.5 Flash e Pro ottimizza kernel per GPU AMD a colpi di tentativi misurati. Paper al workshop ES-FoMo.

Olimpiadi • 2025

Sistemi di Google DeepMind e OpenAI hanno raggiunto il livello da medaglia d'oro all'IMO e alla finale mondiale ICPC.

PER VOI

Cosa serve per progettarli

Scomporre

dividere il lavoro tra agenti con compiti chiari

Stimare la complessità

sapere cosa chiedere e cosa rifiutare

Brute force e stress test

costruire il giudice che non si lascia convincere

Casi limite

trovare il punto esatto in cui l'agente sbaglia

È esattamente ciò per cui vi siete allenati.

PER CONTINUARE

Da provare a casa

Il codelab di questa lectio

[link al codelab] · dieci passi con Antigravity CLI

adk.dev

documentazione di ADK 2.x

Codelab ADK 2 Orchestration

grafi, workflow dinamici e collaborativi

training.olinfo.it

i problemi su cui allenare anche i vostri agenti



Grazie.

E buona premiazione.

Nicola Guglielmi